

Model Danych i Kolekcja (C#)

Definicja klasy reprezentującej kontakt oraz struktury wyliczeniowej:

```
public class Contact
{
    public string? Name { get; set; }
    public string? Surname { get; set; }
    public string? Email { get; set; }
    public string? Phone { get; set; }
    public Gender Gender { get; set; }

    public Contact() { }
}

public enum Gender
{
    Male,
    Female
}
```

Deklaracja kolekcji powiadamiającej o zmianach, która po przypisaniu do kontekstu danych synchronizuje się z interfejsem:

```
public partial class MainWindow : Window
{
    private ObservableCollection<Contact> Contacts { get; set; }

    public MainWindow()
    {
        InitializeComponent();
        Contacts = new ObservableCollection<Contact>();
        DataContext = this;
    }
}
```

Konstrukcja Głównego Widoku (XAML)

Wydzielenie przestrzeni roboczej za pomocą kontenera siatki:

```
<Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*/>
</Grid.RowDefinitions>
```

Zagnieżdżenie paska nawigacyjnego (Menu) w górnym rzędzie oraz podpięcie zdarzeń kliknięcia (Click):

```
<Menu Grid.Row="0">
    <MenuItem Header="File">
        <MenuItem Header="Import" IsEnabled="False" />
        <MenuItem Header="Export" IsEnabled="False" />
        <Separator />
        <MenuItem Header="Exit" Click="MenuItem_Exit" />
    </MenuItem>
</Menu>
```

```

</MenuItem>
<MenuItem Header="Contacts">
    <MenuItem Header="Add contact" Click="MenuItem_AddContact" />
    <MenuItem Header="Clear contacts"
Click="MenuItem_ClearContacts" />
</MenuItem>
<MenuItem Header="About" Click="MenuItem_About" />
</Menu>

```

Konstrukcja DataGrid pozwalająca na automatyczne mapowanie właściwości obiektów na kolumny oraz implementująca natywne dodawanie wierszy:

```

<DataGrid ItemsSource="{Binding}" AutoGenerateColumns="True"
CanUserAddRows="True" />

```

Deklaratywna Logika Widoku - Zasoby (XAML)

Rzutowanie stałych z typu wyliczeniowego Enum na kolekcję zdatną do podłączenia w interfejsie graficznym:

```

<Window.Resources>
    <ObjectDataProvider x:Key="GenderList" MethodName="GetValues"
        ObjectType="{x:Type System:Enum}">
        <ObjectDataProvider.MethodParameters>
            <x:Type TypeName="local:Gender"/>
        </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
</Window.Resources>

```

Mechanizm DataTrigger wymuszający zmianę tekstury docelowej elementu obrazkowego w zależności od wybranego wariantu płci:

```

<DataTemplate x:Key="ContactListItemTemplate" DataType="ListBoxItem">
    <Grid HorizontalAlignment="Stretch">
        <Image Stretch="Fill">
            <Image.Style>
                <Style TargetType="{x:Type Image}">
                    <Style.Triggers>
                        <DataTrigger Binding="{Binding Gender}"
Value="Female">
                            <Setter Property="Source"
Value="{StaticResource WomanAvatar}"/>
                        </DataTrigger>
                        <DataTrigger Binding="{Binding Gender}"
Value="Male">
                            <Setter Property="Source"
Value="{StaticResource ManAvatar}"/>
                        </DataTrigger>
                    </Style.Triggers>
                </Style>
            </Image.Style>
        </Image>
    </Grid>

```

```

        </Grid>
    </DataTemplate>

```

Wykorzystanie AlternationIndex do nałożenia matematycznego modulo na wiersze w liście, cyklicznie przydzielając różne palety kolorystyczne jako tło:

```

<Style x:Key="ContactListItemStyle" TargetType="ListBoxItem">
    <Style.Triggers>
        <Trigger Property="ItemsControl.AlternationIndex" Value="0">
            <Setter Property="Background" Value="#FFAFC5FF" />
        </Trigger>
        <Trigger Property="ItemsControl.AlternationIndex" Value="1">
            <Setter Property="Background" Value="#FF75A1FF" />
        </Trigger>
    </Style.Triggers>
</Style>

```

Ustawienie kontrolki ListBox, aby korzystała ze wstrzykniętych zasobów stylów oraz widoku kolekcji:

```

<ListBox ItemsSource="{Binding Source={StaticResource
ContactsViewSource}}"
    ItemTemplate="{StaticResource ContactListItemTemplate}"
    HorizontalContentAlignment="Stretch"
    ItemContainerStyle="{StaticResource ContactListItemStyle}"
    AlternationCount="2"/>

```

Cykl Życia Drugiego Okna (C# i XAML)

Natychmiastowe wiązanie wartości z interfejsu (formularza) do właściwości obiektu C# poprzez flagę UpdateSourceTrigger=PropertyChanged:

```

<TextBox Grid.Column="1" Grid.Row="0" Height="24"
    Text="{Binding Name, UpdateSourceTrigger=PropertyChanged}"/>

```

Podłączenie rzutowanego typu wyliczeniowego GenderList jako źródła dla wyboru elementów w kontrolce rozwijanej ComboBox:

```

<ComboBox Grid.Column="1" Grid.Row="4" Height="24"
    ItemsSource="{Binding Source={StaticResource GenderList}}"
    SelectedItem="{Binding Path=Gender}" />

```

Kod sterujący asynchronicznym wstrzymaniem wykonania programu do momentu powrotu z modalnego okna wprowadzania danych, wraz z zaciemnieniem okna głównego poprzez operację na alfy (Opacity):

```

private void MenuItem_AddContact(object sender, RoutedEventArgs e)
{
    Opacity = 0.5;

    var addContactWindow = new AddContactWindow();
    if (addContactWindow.ShowDialog() == true)
    {
        Contacts.Add(addContactWindow.NewContact);
    }
}

```

```
    }  
    Opacity = 1;  
}
```